

RFC-0004

Investigation Engine Specification

Version: Draft 0.1

Status: Experimental

Author: Searchlores Academy

Abstract

This document defines the Investigation Engine.

The Investigation Engine is the execution layer of Searchlores.

Its purpose is to transform:

- Subjects
- Lores
- Questions
- Evidence

into:

- Investigations
- Search Maps
- Inquiry Reports

The engine SHALL remain domain-agnostic.

Knowledge belongs to Lores.

Execution belongs to the Engine.

1. Core Principle

Searchlores does not execute prompts.

Searchlores executes investigations.

The fundamental unit of execution is:

Investigation Session

2. Investigation Session

Definition:

A temporary execution context.

Contains:

```
session:
  id:
  subject:
  lore:
  map:
  report:
  state:
```

3. Session Lifecycle

Every investigation SHALL follow:

```
INITIALIZED
  ↓
ANALYZED
  ↓
EXPANDED
  ↓
MAPPED
  ↓
REPORTED
```

↓
COMPLETED

4. State Definitions

INITIALIZED

Subject loaded.

Lore loaded.

No analysis yet.

ANALYZED

Assumptions identified.

Vectors generated.

Questions created.

EXPANDED

Plugins executed.

Additional knowledge generated.

MAPPED

Graph constructed.

Relationships established.

REPORTED

Human-readable output generated.

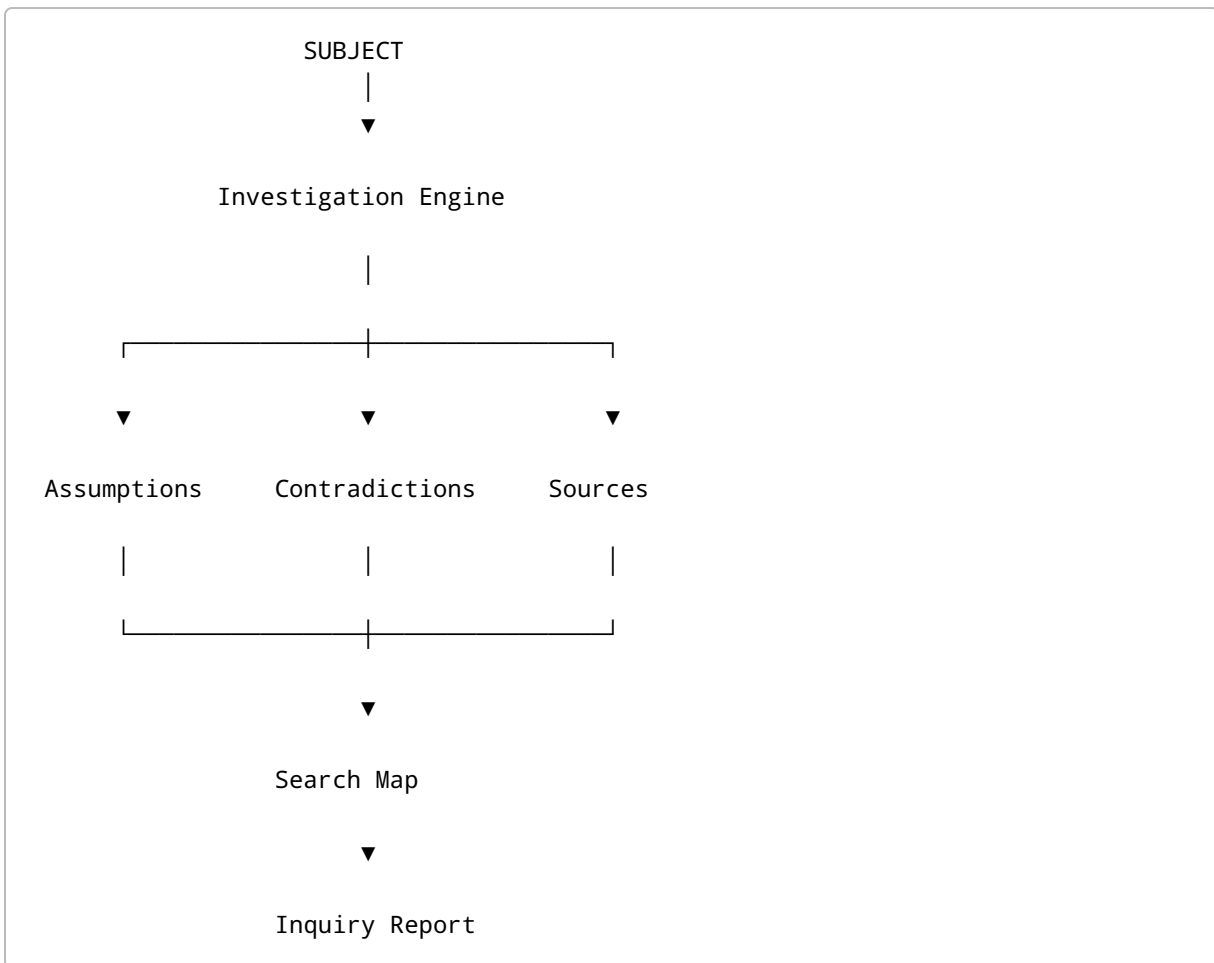
COMPLETED

Session finalized.

Artifacts stored.

5. Engine Architecture

Reference architecture:



6. Pipeline Model

The engine SHALL use a pipeline architecture.

Definition:

A sequence of processors operating on an Investigation.

Example:

```
Subject
  ↓
Assumption Cracker
  ↓
Mythology Scanner
  ↓
Contradiction Hunter
  ↓
Source Archaeologist
  ↓
Search Mapper
```

7. Investigation Context

Definition:

Shared execution memory.

Structure:

```
context:
  investigation:
  lore:
  plugins:
```

```
metadata :
```

Every plugin receives the same context.

8. Processor

Definition:

A unit of transformation.

Interface:

```
class Processor:

    def process(
        self,
        context
    ):
        pass
```

9. Plugin

Definition:

A dynamically loaded processor.

Requirements:

Every plugin MUST:

- declare a name
- declare a version
- implement process()

Example:

```
class AssumptionCracker:

    name = "assumption_cracker"

    version = "1.0"

    def process(self, context):
        pass
```

10. Plugin Categories

Official categories:

```
analysis
archaeology
mapping
reporting
export
validation
```

11. Analysis Plugins

Purpose:

Generate investigation content.

Examples:

```
AssumptionCracker
MythologyScanner
ConsensusDetector
```

NarrativeMapper

12. Archaeology Plugins

Purpose:

Reverse engineer artifacts.

Examples:

PromptArchaeologist

SourceArchaeologist

NarrativeArchaeologist

ConceptArchaeologist

13. Mapping Plugins

Purpose:

Construct Search Maps.

Examples:

GraphBuilder

ContradictionMapper

SourceTreeBuilder

14. Reporting Plugins

Purpose:

Generate outputs.

Examples:

MarkdownReporter

HtmlReporter

YamlReporter

15. Export Plugins

Purpose:

Export investigations.

Examples:

SVGExporter

GraphvizExporter

MermaidExporter

GephiExporter

16. Validation Plugins

Purpose:

Verify integrity.

Examples:

LoreValidator

MapValidator

SessionValidator

17. Event System

The engine SHALL expose events.

Events allow plugins to react.

Official events:

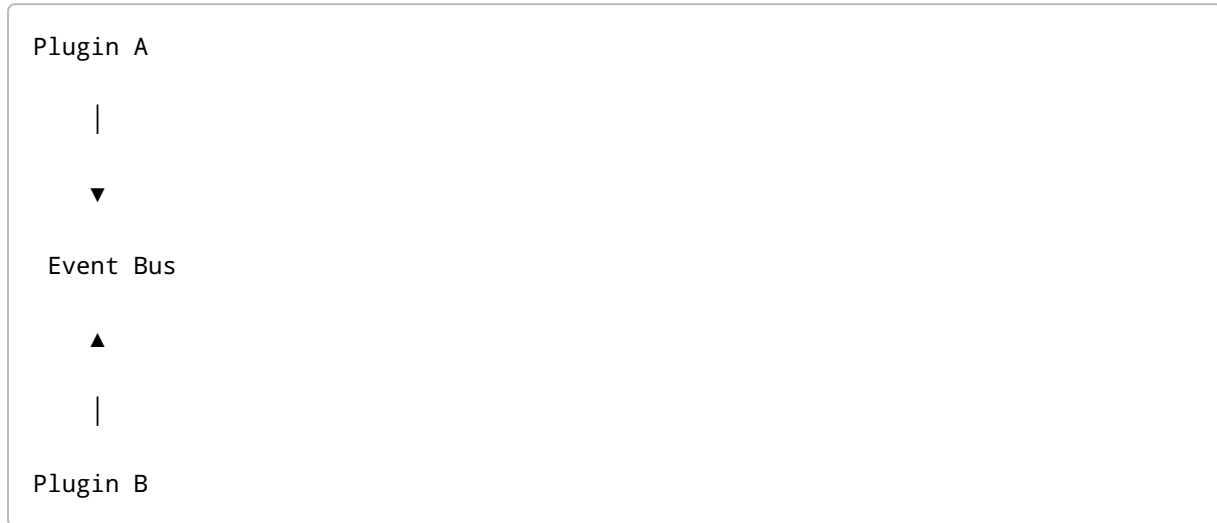
```
session_started
subject_loaded
lore_loaded
analysis_completed
map_generated
report_generated
session_completed
```

18. Event Structure

```
event:
  name:
  timestamp:
  payload:
```

19. Event Bus

Reference model:



Plugins SHOULD communicate through events.

Plugins SHOULD NOT directly depend on each other.

20. Dependency Model

Plugins MAY declare dependencies.

Example:

```
plugin:
  name:
  depends_on:
    - assumption_cracker
    - mythology_scanner
```

21. Plugin Ordering

Execution order SHALL be determined by:

1. Dependencies
2. Priority
3. Registration order

22. Search Map Generation

The engine MUST generate:

Nodes

Edges

Relationships

from Investigation data.

Search Maps are generated after analysis.

23. Report Generation

Reports are generated after mapping.

Reports SHALL summarize:

- assumptions
- vectors
- contradictions
- narratives
- questions

24. Lore Execution

A Lore acts as:

Execution Blueprint

The engine reads:

- assumptions
- myths
- vectors
- plugins

and creates the investigation pipeline.

25. Investigation Modes

Official modes:

exploration

archaeology

audit

comparison

mapping

26. Exploration Mode

Goal:

Expand understanding.

Produces:

- vectors
 - questions
 - maps
-

27. Archaeology Mode

Goal:

Expose hidden structures.

Produces:

- assumptions
 - origins
 - narratives
-

28. Audit Mode

Goal:

Challenge claims.

Produces:

- contradictions
 - evidence gaps
 - source chains
-

29. Comparison Mode

Goal:

Compare multiple subjects.

Produces:

- overlap maps
 - divergence maps
-

30. Mapping Mode

Goal:

Generate topology.

Produces:

- Search Maps
- Source Trees
- Contradiction Graphs

31. Storage Layer

The engine SHOULD remain storage-agnostic.

Supported targets MAY include:

YAML

JSON

SQLite

PostgreSQL

Graph Databases

32. Knowledge Persistence

Investigation results MAY be stored.

Artifacts:

.investigation

.map

.report

33. Security Model

Plugins SHALL execute in isolation.

Plugins SHOULD NOT modify:

- core engine state
- unrelated investigations

without explicit permission.

34. Extensibility Principle

The engine MUST be expandable without modifying core code.

New capabilities SHOULD be implemented as plugins.

35. Philosophy

The Investigation Engine is not a reasoning engine.

It is not an oracle.

It is not a knowledge base.

It is an expedition coordinator.

The engine does not provide truth.

The engine organizes exploration.

Truth remains the responsibility of the investigator.

Conclusion

Searchlores executes investigations.

Investigations generate maps.

Maps reveal structures.

Structures guide inquiry.

The Search Is The Program.